University Number: _____

THE UNIVERSITY OF HONG KONG

FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE

COMP7104/DASC7104 - Advanced Database Systems

Date: December 13, 2023 **Time: 6:30pm-8:30pm**

<INSTRUCTION ABOUT NUMBER OF QUESTION TO BE ANSWERED: OVERALL AND FROM EACH SECTION>
23 questions overall, divided into two sections as follows:
Section 1 – Multiple-choice questions – 16 questions
Section 2 – Detailed-answer questions – 8 questions

<INDICATION OF COMPULSORY QUESTIONS>
None.

<INFORMATION ABOUT THE MARK VALUE OF QUESTIONS>
Section 1 – 1.5 points per question – 24/100 points in total
Section 2 – number of points depends on the question – 76/100 points in total

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

Candidates are permitted to bring one sheet of A4-sized paper with printed or written notes on both sides to the examination.

Mark your University Number on each page (top right corner).

University Number: _____

Brand and Type of Calculator: _____

General indications

- For questions involving arithmetic expressions, numerical work or derivations, credit will be given only if the work leading to the answer is shown.

Section 1 – Multiple-choice questions – 16 questions – 1.5 point per question

Circle the true answer(s) to each question. Each question may have zero, one, or several valid answers. For each question, only the identification of all and only the valid answers will allow you to obtain the assigned point.

Q1.1 Which of the following assertions about join algorithms are true?

(A) In a page nested loop join (PNLJ) where both tables fit entirely in memory, the choice of which table to use as the inner/outer table may affects I/O cost.

(B) In a page nested loop join (PNLJ) where one of the tables fits entirely in memory, it is better to use that table as the inner one.

(C) In an index nested loop join (INLJ) where only one of the tables has an index on the join attribute, the choice of inner/outer branch should be made based on the table sizes.

Explanation: A is false, if both tables are small enough to fit entirely in memory, then they can be read once, so the order would not be important. B is true: having the table that fits in memory as inner would reduce I/O costs as it would then be read only once, instead of repeatedly for each tuple of the outer table. C is false, the choice of the inner/outer tables should be based on which table has the index on the join attribute; only if both tables had an index, then their sizes would have been a factor.

Q1.2 Which of the following assertions about the System R / Selinger algorithm are true?

(A) It never considers plans with cartesian products because they can always be expressed as a join.

(B) It only explores right-deep plans.

(C) It keeps track of interesting orders as they may I/O cost for upstream operations.

Explanation: A is false, sometimes the query requires a carthesian product, and cannot be expressed as a join. B is false (left deep), C is true as discussed in class.

Q1.3 Which of the following assertions about left-deep plans are true?

(A) Two left-deep plans can differ in the order of relations and produce the same output.

(B) Two left-deep plans can differ in the access method for each leaf operator and produce the same output.

(C) Two left-deep plans can differ in the join method for each join operator and produce the same output.

Explanation: all true as discussed on class.

Q1.4 Which of the following is NOT a good rule for rewriting and optimizing relational algebra plans ?

(A) Push projection below the right (inner) branch of a CNLJ (Chunk Nested Loop Join).

(B) Push selection below the right (inner) branch of a CNLJ (Chunk Nested Loop Join).

(C) Push selection below the left (outer) branch of a CNLJ (Chunk Nested Loop Join).

Explanation : B is not a good rule, as discussed in class, both A and C are useful .

Q1.5 Which of the following join operators is a pipeline breaker ?

(A) Index Nested Loop Join.

(B) Sort Merge Join.

(C) Grace Hash Join.

Explanation : B and C, they need to see the entire input of the outer branch (to sort / hash) it. An output of the outer branch child operator can be streamed as input to O one tuple (or batch of tuples) at a time

Q1.6 Which assumptions are followed in the selectivity estimator of the System R / Selinger algorithm?

(A) Independence of predicates.

(B) Uniform distribution within histogram bins (when histograms available).

(C) Normal distribution for integer attributes.

Explanation: A and B true as discussed, C is not something we consider.

Q1.7 Which of the following assertions are true about transaction management in RDBMSs?

(A) All serial schedules are also serializable.

(B) Interleaving transactions is always faster than doing them one at a time.

(C) All conflict-serializable schedules are also recoverable.

Explanation:

A is obviously true, by the definition. For B, there could be no conflicts and since interleaving has a significant overhead cost, interleavingcould actually run slower. C is false as seen in class.

Q1.8 Which of the following properties are true about the 2PL / strict 2PL protocol?

(A) strict 2PL enables less schedules than 2PL.

(B) strict 2PL requires that transactions acquire locks gradually, and release them all all at once at the end of the transaction.

(C) strict 2PL ensures that all schedules are recoverable.

Explanation: All true, A since strict 2PL has more restrictive rules, B by definition, C as discussed in class.

Q1.9 Which of the following properties are true about the 2PL / strict 2PL protocol?

(A) Some conflict serializable schedules cannot be produced when using 2PL.

(B) Schedules that are conflict-serializable will not produce a cyclic dependency graph.

(C) Both strict 2PL and 2PL enforce conflict serializability.

Explanation: A: 2PL enforces conflict serializability but may not allow all conflict serializable schedules (e.g. W1(X), R2(X), W1(Y), R2(Y) is impossible under 2PL). For B, a schedule is conflict serializable if and only if the dependency graph is acyclic, so conflict serializable implies an acyclic dependency graph. For C, because you cannot get any new locks after releasing a lock in both strict and non-strict 2PL, the dependency graph for the resulting schedule can never be cyclic, so conflict serializable.

Q1.10 Which of the following assertions are true about data partitioning and query evaluation in PDBMSs?

(A) If we partition data using the round robin scheme, look-up by key will incur a higher cost than with range or hash partitioning.

(B) If we partition data using the range scheme, look-up by key will incur a higher cost than using the hash scheme.

(C) A join over two tables that requires a symmetric shuffle will incur fewer network I/Os than one that requires an asymmetric shuffle.

Explanation:

A is true, since we have to broadcast, B is false, same cost, C is False, it is the other way around.

Q1.11 We know that 50% of the records in table Person in our database all share the same value for the attribute *name*. Assume we run a series of experiments, each time increasing the number of machines over which we hash-partition the Person table *by name*, and measure the time to complete a parallel scan of this table each time. Across experiments, the completion time for parallel scan will be a function of the number of machines used.

(A) Constant.

(B) Exponential.

(C) Linear.

Explanation: Constant. Person is hash-partitioned on name, so a single machine will get every record with the popular name. As we increase the number of machines, we remain bottlenecked by the single machine with the popular name (which always has to scan half the records), so the completion time is constant.

Q1.12 Which of the following assertions are true about data partitioning and query evaluation in PDBMSs?

(A) If data is partitioned using round robin, when searching for a record, we have to perform broadcast lookup to all the machines.

(B) Range partitioning is the only partitioning scheme that ensures balanced load, i.e., each node gets a similar amount of data.

(C) Round-robin partitioning is the only partitioning scheme that ensures balanced load, i.e., each node gets a similar amount of data.

Explanation:

A is true, B is false since C is true.

Q1.13 Which of the following assertions are true about PDBMS query evaluation?

(A) Pipeline parallelism scales up with the amount of data.

(B) Partition parallelism scales up with the amount of data.

(C) Pipeline parallelism scales up to the pipeline depth.

Explanation: A is false: scales to the depth of the pipeline. B is true, C is true.

Q1.14 Which of the following assertions are true about PDBMS query evaluation?

(A) There is no extra disk I/O cost for parallel Grace Hash Join when we re-partitioning (shuffling) the data across machines.

(B) Data skew may be a problem regardless the partitioning scheme.

(C) Parallel aggregates require that we compute the same aggregation function locally then globally.

Explanation: A is true: tuples are streamed from the full table scans into the local hash join operators on each of the machines, so the repartitioning does not incur any additional disk I/O cost (there is a network cost), but no additional disk I/O cost). B is false, round-robin does not suffer from data skew. C) is false, the aggregation function may be different locally / globally.

Q1.15 Which of the following are valid reasons to replicate data in NoSQL database ?

(A) To handle ever-increasing database sizes.

(B) To recover from failures.

(C) To enable workload scalability.

Explanation: A is false: that's what partitioning is for. B is true, is a version of Availability as well, C is true, as discussed.

Q1.16 Which of the following assertions are true for NoSQL database systems ?

(A) They were designed to cope with high rates of simple read/write operations on relational data.

(B) They were designed to cope with long running read-only analytical queries.

(C) They were designed for mixed analytical / transactional workloads.

Solution: A is false, high rate of simple read/write but not necessarily for relational data, B and C are false as discussed.

Section 2 – Detailed-answer questions – 8 questions – 76 points in total

Q2.1 – (Join Algorithms) (11 points) – Consider relations Alfa(a, c), Beta(a, b, e), and Gamma(a, d, f) on which we want to perform a natural join (join by equality on the common attribute a). No indexes are available on these relations.

- There are B = 750 buffer pages (main-memory cache)
- Table Alfa has M = 2,500 pages with 80 records per page
- Table Beta has N = 600 pages with 300 records per page
- Table Gamma has O = 1,500 pages with 150 records per page
- The join result of Alfa and Beta has P = 500 pages

What is the **I/O cost** for the following joins algorithms, ignoring the cost of the final writing to disk of join results?

(a) (1 point) Page nested loop join (PNLJ) with Alfa as the outer relation and Beta as the inner relation?

(b) (1 point) Chunk nested loop join (CNLJ) with Gamma as the outer relation and Beta as the inner one?

(c) (1 point) Chunk Nested Loop Join (CNLJ) with Beta as the outer relation and Gamma as the inner one?

In what follows, let us assume that we compute a Sort-Merge Join (SMJ) with Alfa as the outer relation and Gamma as the inner relation.

(d) (1 point). What is the cost of sorting the records in Alfa on attribute a?

(e) (1 point) What is the cost of sorting the records of Gamma on attribute a?

(f) (1 point) What is the cost of the merge phase in the worst-case scenario of the previous SMJ?

(g) (1 point) What is the cost of the merge phase assuming there are no duplicates in the join attribute a?

(h) (1 point) Let us assume we first join Alfa and Beta, and then join the result with Gamma. What is the cost of the final merge phase assuming there are no duplicates in the join attribute a?

Let us consider next a hash-join with Beta as the outer relation and Gamma as the inner one:

(i) (1 point) Assuming no recursive partitioning is needed, detail the cost of the probing (conquer) stage.

(j) (1 point) Assuming no recursive partitioning is needed, detail the cost of the partitioning stage.

(k) (1 point) Assume that the tables do not fit in main memory and that a large number of distinct values will hash to the same bucket using hash function h_p. What should we do in this case?

Explanation:

(a) M + m × N = 2500 + 2500 × 600 = 1 502 500

(b) O + ⌈ O/B−2⌉ × N = 1500 + ⌈1500 / 748 ⌉ × 600 = 1500 + 1800 = 3300

(c) N + ⌈ N/B−2⌉ × O = 600 + ⌈600/748⌉ × 1500 = 600 + 1500 = 2100

(d) nb of passes = 1 + ⌈$\log_{B-1}$(⌈M/B⌉)⌉ = 1 + ⌈$\log_{749}$(⌈2500/750 ⌉)⌉ = 1 + 1 = 2, so 2M × passes = 2 × 2500 × 2 = 10000

(e) nb of passes = 1 + ⌈$\log_{B-1}$(⌈O/B⌉)⌉ = 1 + ⌈$\log_{749}$(⌈1500/750 ⌉)⌉ = 1 + 1 = 2, so 2 × O × passes = 2 × 1500 × 2 = 6000

(f) M × O = 2500 × 1500 = 3 750 000

(g) M + O = 2500 + 1500 = 4000

(h) P + O = 500 + 1500 = 2000

(i) (N + O) = (600 + 1500) = 2100

(j) 2 × (N + O) = 2 × (600 + 1500) = 2 × 2100 = 4200

(k) Use Grace Hash join, that is rehash by another hash function the too large partitions from the first partitioning stage.

Q2.2 – Grace Hash Join (8 points) – Products has a total of 100 pages and 20 tuples per page. Orders has a total of 50 pages and 50 tuples per page. Assume the hash functions uniformly distribute the data for both tables.

(a) (2 points) If we had 10 buffer pages, how many partitioning phases would we require for Grace Hash Join? Consider which table we should build the hash table in the probing phase on.

(b) (2 points) What is the I/O cost for the Grace Hash Join in that case?

(c) (2 points) For the above question, if we only had 8 buffer pages, how many partitioning phases would be necessary?

(d) (2 points) What will be the I/O cost in that case?

Explanation: (a) Orders is smaller, so we need its partitions to be at most B - 2 = 8 pages. After 1 partitioning pass, we have partitions of size 6, which is ≤ 8 so we only need 1 partitioning pass.

(b) We need 1 partitioning pass.
Partitioning phase:
ceil([P]/(B - 1)) = 12 pages per partition for P, 12(9) pages in total after partitioning ceil([O]/(B - 1)) = 6 pages per partition for Orders, 6x9 pages in total after partitioning
Partitioning IOs: 100 I/Os to read from Products + 12x9 to write for Products + 50 I/Os to read from Orders + 6x9 to write for Orders = 312 I/Os
Note: in this example it is assumed that all the partitions are of the same size, even if they will not be entirely full. Hence the cost which is 100 (read) +12*9 (write) for one partitioning stage of the Products, instead of 100 (read) + 100 (write); both answers were accepted, 100 + 100 and 100+12*9.

Probing phase: 12x9 + 6x9 = 162 I/Os to read from Products and Orders
Total: 312 + 162 = 474 I/Os

(c) Orders is smaller, so we need its partitions to be at most B - 2 = 6 pages. After 1 partitioning pass, we have partitions of size 8, which is too big to fit in B-2 buffer pages. We need a second partitioning pass. 8 / 7 = 1.1 so 2 pages, which is small enough to fit in B-2 buffer pages. Therefore, we need 2 passes in total.
(d) Partitioning phase:
ceil([P]/(B - 1)) = 15 pages per partition for P
ceil([O]/(B - 1)) = 8 pages per partition for O
ceil([P]/(B - 1)) = 3 pages per partition for second pass for P
ceil([O]/(B - 1)) = 2 pages per partition for second pass for O
Partitioning IOs: [100 + 50] (1st read) + [15x7 + 8x7] (1st write) + [15x7 + 8x7] (2nd read) + [3x49 + 2x49] (2nd write) = 717 I/Os
Build and Probe Phase: 3x49 + 2x49 = 245 IOs
Total: 717 + 245 = 962 I/Os

Q2.3 – Query Optimization – Selectivity Estimation (8 points). Consider the following schema:

HORSE(horseId INTEGER PRIMARY KEY, breed VARCHAR(50), age INTEGER);

CAVALIER(cavId INTEGER PRIMARY KEY, name VARCHAR(50), age INTEGER);
RIDE(horseId INTEGER REFERENCES Horse(horseId), cavId INTEGER REFERENCES Cavalier(cavId), date DATE, PRIMARY KEY (horseId, cavId));

We make the following assumptions about the distribution of data:

- $15 \leq$ Cavalier.age ≤ 90
- $5 \leq$ Horse.age ≤ 50
- cavId has 1000 distinct values
- horseId has 1000 distinct values
- breed has 10 distinct values
- we assume attribute independence over all attributes

Consider the following query:

```
SELECT C.name
FROM Horse H, Cavalier C, Ride R
WHERE H.horseId = R.horseId AND C.cavId = R.cavId
AND (C.age < 30 OR H.breed = "Mustang") AND H.age >= 30;
```

Compute the selectivity for each of the following predicates from the WHERE clause. Write only your final answer, as a simple fraction.

(a) (2 points) *H.horseId = R.horseId*

Solution: 1/1000

Explanation:

Selectivity = 1 / MAX(NKeys(H.horseId), NKeys(R.horseId))

NKeys(H.horseId) = 1000

NKeys(R.horseId) $\leq$ 1000 because of the foreign key referencing H.horseId

1/max(1000, x) = 1000, since x $\leq$ 1000.

(b) (2 points) *C.cavId = R.cavId*

Solution: 1/1000

Explanation:

Selectivity = 1 / max(NKeys(C.cavId), NKeys(R.cavId))

NKeys(U.cavId) = 1000

NKeys(R.cavId) <= 1000 because of foreign key referencing C.cavId

So 1/Max(1000, x) where x <= 1000 = 1/1000.

(c) (2 points) H.age >= 30

Solution: 0.45

Explanation: Selectivity = (High(H.age)-30)/(High(H.age)-Low(H.age) + 1) + 1/ number distinct values of H.age = (50 -30) / (50-5+1) + 1/45 (since we have >=) = 20/46 + 1/45 = approx.. 0.45

The condition is like (age>30) OR (age=30) hence the two terms in the RF formula. We are not applying the condition > 29, but the condition >=30, even if the age value is an integer.

(d) (2 points) C.age < 30 OR H.breed = “Mustang”

Solution: 14/50 = 7/25=0.28

Explanation: Selectivity= (approx. 15/75 so 1/5) + 1/10 - 1/5 x 1/10 = 14/50

Q2.4 – Locking, 2PL & strict 2PL (5 points) – Consider the following schedule, where time flows top to bottom, and Lock means a request for the lock, in either shared (S) or exclusive (X) mode, modifying resources B and F.

T1	T2
Lock_X(B)	
Read(B)	
B := B * 10	
Write(B)	
Lock_X(F)	
Unlock(B)	
	Lock_S(F)
F := B * 100	
Write(F)	
Commit	
Unlock(F)	
	Read(F)
	Unlock(F)
	Lock_S(B)
	Read(B)
	Print(F + B)
	Commit
	Unlock(B)

(a) (1 point) What is printed, assuming we initially have B = 3 and F = 300?

(b) (1 point) Does the execution use 2PL, strict 2PL, or neither?

(c) (1 point) Would moving Unlock(F) in the second transaction to any point after Lock_S(B) make it become 2PL (or keep it 2PL if it was already so) ?

(d) (1 point) Would moving Unlock(F) in the first transaction and Unlock(F) in the second transaction to

the end of their respective transactions change this (or keep it) in strict 2PL?

(e) (1 point) Would moving Unlock(B) in the first transaction and Unlock(F) in the second transaction to the end of their respective transactions change this (or keep it) in strict 2PL?

Explanation:

a) 3030

b) Neither - T2 unlocks S(F) before it acquires S(B)

c) Yes - all locks would be acquired (for T2) before any are released.

d) No - T1 still unlocks B before the end of the transaction

e) Yes - all unlocks would only happen when the respective transactions end

Q2.5 – Lock manager (9 points) – The lock manager answers three kinds of requests:

- request(transaction, resource, mode): this is to allow a transaction to request one lock, in either Shared (S) or eXclusive (X) mode. The manager grants the request if (when) there is no queue and the request is compatible with existing locks. Otherwise, it should queue the request (at the back of the queue) and block the transaction.
- release(transaction, resource): this allows a transaction to release one lock that it holds.
- upgrade(transaction, resource): this allows a transaction to upgrade a held lock from Shared to Exclusive. Such a request has priority over any existing queued requests (it is placed in the front of the queue if it cannot proceed).

In this context, we will have 3 transactions (T1, T2, T3) and 2 resources that they will try to lock (A and B). After each lock request is processed, fill out the following two tables that store the information for what locks are held:

The following lock requests are made in this order:

a) (1 point) request(T1, A, X)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

b) (1 point) request(T2, B, S)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

c) (1 point) request(T3, B, X)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

d) (1 point) request(T3, A, S)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

e) (1 point) release(T1, A)

Transaction	Locks held

T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

f) (1 point) request(T2, A, S)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

g) (1 point) request(T1, B, S)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

h) (1 point) upgrade(T2, B, X)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

i) (1 point) release(T1, B)

Transaction	Locks held
T_1	
T_2	
T_3	

Resource	Locks	Queue
A		
B		

Solution:

(a) request(T1, A, X) There are no locks held, so this request will be immediately granted.

Transaction	Locks held
T_1	X(A)
T_2	
T_3	

Resource	Locks	Queue
A	T1 X(A)	
B		

(b) request(T2, B, S) No locks on B so this request can be granted immediately.

Transaction	Locks held
T_1	X(A)
T_2	S(B)
T_3	

Resource	Locks	Queue
A	T1 X(A)	
B	T2 S(B)	

(c) request(T3, B, X) T3 cannot get its request

Transaction	Locks held
T_1	X(A)
T_2	S(B)
T_3	

Resource	Locks	Queue
A	T1 X(A)	
B	T2 S(B)	Request(T3, B, X)

(d) request(T3, A, S) T3 cannot get its request

Transaction	Locks held
T_1	X(A)
T_2	S(B)
T_3	

Resource	Locks	Queue
A	T1 X(A)	Request(T3, A, S)
B	T2 S(B)	Request(T3, B, X)

(e) release(T1, A) Release of X lock on A by T1, T3 gets its request S

Transaction	Locks held
T_1	
T_2	S(B)
T_3	S(A)

Resource	Locks	Queue
A	T3 S(A)	
B	T2 S(B)	Request(T3, B, X)

(f) request(T2, A, S) Request granted

Transaction	Locks held
T_1	
T_2	S(B), S(A)
T_3	S(A)

Resource	Locks	Queue
A	T2 S(A) T3 S(A)	
B	T2 S(B)	Request(T3, B, X)

(g) request(T1, B, S) request granted

Transaction	Locks held

T_1	S(B)
T_2	S(B), S(A)
T_3	S(A)

Resource	Locks	Queue
A	T2 S(A) T3 S(A)	
B	T2 S(B) T1 S(B)	Request(T3, B, X)

(h) upgrade(T2, B, X) – cannot be granted since T1 has the shared lock on B, placed in front of queue

Transaction	Locks held
T_1	S(B)
T_2	S(B), S(A)
T_3	S(A)

Resource	Locks	Queue
A	T2 S(A) T3 S(A)	
B	T2 S(B) T1 S(B)	Upgrade(T2, B, X), Request(T3, B, X)

(i) release(T1, B)

Transaction	Locks held
T_1	
T_2	X(B), S(A)
T_3	S(A)

Resource	Locks	Queue
A	T2 S(A) T3 S(A)	
B	T2 X(B)	Request(T3, B, X)

Q2.6 - Concurrency control (8 points) Consider the following schedule (time flows top-to-bottom, each line corresponds to one clock “tick”). R stands for Read, W stands for Write, Com stands for Commit.

	T1	T2	T3	T4
1	R(A)			
2		R(A)		
3			R(C)	
4			W(C)	
5		R(B)		
6		W(B)		
7	R(B)			
8				R(B)
9	R(C)			
10				R(C)
11				W(B)
12		COM		
13			COM	
14	COM			
15				COM

(a) (1 point) Is this schedule serial?

(b) (2 points) Draw the conflict graph for this schedule.

(c) (1 point) Is this schedule conflict serializable?

(d) (2 points) Which of the two locking protocols learned in class could have produced this schedule?

(e) (2 points) Which of the following schedules below are conflict equivalent to the schedule above?

A. T3, T1, T2, T4

B. T2, T3, T1, T4

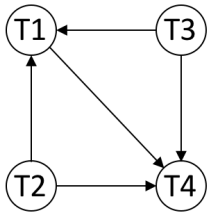
C. T4, T3, T1, T2

D. T3, T2, T1, T4

Explanation:

(a) obviously no

(b)



(c) yes, no cycle

(d) Two Phase Locking (not strict): 2PL is possible because each transaction could release its lock immediately after it's done with its operation and so this schedule would be allowed to happen. Strict 2PL is not possible because transactions have to hold their locks (ex. T1 would not be able to R(B) in 7 because T2 would have to hold its exclusive lock on B until it ends)

(e)

~~A. T3, T1, T2, T4~~

B. T2, T3, T1, T4

~~C. T4, T3, T1, T2~~

D. T3, T2, T1, T4

Just by doing a topological sort, we know that T4 has to be last and T1 has to be second to last. Whether T2 or T3 comes first is irrelevant since they have no incoming edges, so we have 2 possible conflict equivalent schedules.

Q2.7 – PDBMS (12 points) Assume that you have access to the following two relations:

Dungeon(dungeonId, name, location, surface, owner)

Dragon(dragonId, name, type, weight)

The main parameters for this question are summarized in the table below:

Parameter	Symbol	value
Number of nodes	M	4
Size of page (KB)	s	4
Pages in RAM per node	B	8
Size of Dungeon	$[D_g]$	128
Size of Dragon	$[D_r]$	4
Time for each I/O (ms)	t	5

Assume we have 4 nodes, each with 8 pages in memory for joins. We want to measure time, assuming that an I/O takes 5ms. For the following questions, when we ask for execution time, we are only concerned with the time associated with I/Os (e.g., CPU and other costs are negligible). Assume that we can send individual records over the network with no overhead in terms of network cost.
The first two questions will deal with the following query:

```
SELECT DG.name, DR.name, DG.surface, DR.type
FROM Dungeon DG, Dragon DR
WHERE DG.owner = DR.name;
```

(a) (2 points) Assuming the Dungeon and Dragon relations are both round-robin partitioned across the 4 nodes, what is the largest amount of data we ship across the network, in KB, of the query above, assuming we want to execute a Sort Merge Join?

Solution: s * ([D_g] + [D_r]) = 4KB * (128 + 4) = 528 KB – we will, in the worst case, have to send over every single tuple because we need to *range partition* for sort-merge.

(b) (2 points) Under the same assumption as before, what would be the expected amount of data we ship across the network, assuming uniform distribution of data by the round-robin partition with respect to the domain of the owner / name attributes.

Solution: roughly ¼ of the data is already at the right node, so will not be shipped, so s * ([D_g] + [D_r]) * 3/4 = 3 * (128 + 4) = 396 KB

(c) (2 points) Assuming the Dungeon relation starts out hash-partitioned on the "location" key across the 4 machines, and that 75% of Dungeon gets mapped to one machine, how long will it take to complete a parallel scan of Dungeon?

Solution: t * 0.75 * [D_g] = 5 * 0.75 * 128 = 5 * 96 = 480 ms, the machine with the most data being the

one determining the overall time (being the slowest).

Suppose we now add another table, with the following schema:

Rider(riderId, name, dragonType, salary)

which has 40,000 pages and is round-robin partitioned across 4 new machines with only this data.

The next two questions deal with the following query:

```
SELECT DR.name, R.name
FROM Dragon DR, Rider R
WHERE R.dragonType = DR.type;
```

(d) (2 point) What is the name of the join strategy that provides the lowest possible network cost (amount of data shipped) to execute the query above? Explain why others would not be a good idea.

Solution: Broadcast join to the machines holding Rider, since one table (Dragon) is very small compared to the other one (Rider). Symmetric shuffle and asymmetric shuffle would cost more because we would have to re-partition Rider.

(e) (2 points) What is the amount of data shipped in KB to execute that join strategy?

Solution: s * 4 * 4 = 4 KB * 4 * 4 = 64 KB (perform a broadcast join - send over the entire Dragon table to each machine)

(f) (2 points) We wish to compute the square root of the sum of the squared salary of each rider, as in:

```
SELECT SQRT(SUM(salary * salary)) FROM Rider
```

Do we need to repartition the data? What would be a global and local aggregate function that would allow us to efficiently compute the result via hierarchical aggregation?

Solution: no need to repartition, we would do locally sum of squared salary, and globally squared root of sums.

Q2.8 – Query optimization – System R / Selinger Algorithm (15 points) For the following questions, assume the following:

- The assumptions about uniformity with attribute domains and independence between attributes.
- We use System R defaults whenever selectivity estimation is not possible.
- Primary key IDs are sequential, starting from 1.
- To simplify, our optimizer does not consider interesting orders.

CREATE TABLE Engineer (eid INTEGER PRIMARY KEY, name VARCHAR(32), field VARCHAR(64)), years_experience INTEGER) ;

- Table Engineer has 25,000 records in 500 pages.
- Index 1: Clustered(field). There are 130 unique fields.
- Index 2: Unclustered(years_experience) and there are 11 unique values in the range [0,10] for this attribute.

CREATE TABLE Application (eid INTEGER REFERENCES Engineer, cid INTEGER REFERENCES Company, status TEXT, referral INTEGER, (eid, cid) PRIMARY KEY)

- Table Application has 100,000 records in 10,000 pages.
- Index 3: Clustered(cid, eid).
- Status has 10 unique values.

CREATE TABLE Company (cid INTEGER PRIMARY KEY, open_roles INTEGER)

- Table Company has 500 records in 100 pages.
- Index 4 : Unclustered(cid).
- Index 5: Clustered(open_roles) and there are 500 unique values in the range [1,500] for this attribute.

We consider the following query:

```
SELECT Engineer.name, Company.open_roles, Application.referral
FROM Engineer, Application, Company
WHERE Engineer.eid = Application.eid.                    -- selectivity 1
AND Application.cid = Company.cid                        -- selectivity 2
AND Engineer.years_experience > 6                        -- selectivity 3
AND (Engineer.field='Naval' OR Company.open_roles <= 50) -- selectivity 4
AND NOT Application.status = 'Pending'                   -- selectivity 5
ORDER BY Company.open_roles;
```

Write the reduction factor (RF) for each clause.

(a) (1 point) *Engineer.eid = Application.eid*

(b) (1point) *Application.cid = Company.cid*

(c) (1 point) *Engineer.years_experience > 6*

(d) (2 points) *(Engineer.field='Naval' OR Company.open_roles <= 50)*

(e) (1 point) *NOT Application.status = 'pending'*

Solution: (a) 1 / max(25000, 25000) = 1/ 25, 000 since there are exactly 25000 values in Engineer.eid, and due to the foreign key, there are at most 25000 values in Application.eid.

(b) 1/ max(500, 500) = 500 since there are exactly 500 values in Company.cid, and due to the foreign key, there are at most 500 values in Application.cid

(c) 10−6 / (10−0+1) = 4/11 We have 11 unique values. Therefore we use the equation for less than or equal to which is (high key - value) / (high key - low key + 1).

(d) (1/130 + 1/10) – (1/130 x 1/10) = 10/1300 +130/1300 – 1/1300 = 139/1300

We can find the selectivity that they are in the Naval field by using the equation 1/distinct values. Next, we find the selectivity that open positions are less than or equal to 50 using the equation (v - low key) / ((high key - low key + 1) + (1 / number distinct)). Lastly we combine these two selectivities using S(p1) + S(p2) - S(p1)S(p2) to determine the selectivity of having one or the other.

(e) 1-1/10 = 9/10, since given 10 unique values, the non-negated predicate has selectivity 1/10 , so we can use the equation for NOT which is 1 - selectivity of the predicate.

(f) (3 points) For each predicate, mark the first pass of Selinger's algorithm that uses its selectivity to estimate output size.

i. Selectivity 1: Pass 1, Pass 2, or Pass 3.
ii. Selectivity 2: Pass 1, Pass 2, or Pass 3.
iii. Selectivity 3: Pass 1, Pass 2, or Pass 3.
iv. Selectivity 4: Pass 1, Pass 2, or Pass 3.
v. Selectivity 5: Pass 1, Pass 2, or Pass 3.

(g) (1.5 points) Mark the choices for all access plans that would be considered in pass 2 of the Selinger algorithm.

i. Engineer JOIN Application (800 IOs)
ii. Application JOIN Engineer (750 IOs)
iii. Engineer JOIN Company (470 IOs)
iv. Company JOIN Engineer (525 IOs)
v. Application JOIN Company (600 IOs)
vi. Company JOIN Application (575 IOs)

(h) (1.5 points) Mark the choices from the previous question for all access plans that would be chosen at the end of pass 2 of the Selinger algorithm.

(i) (1.5 points) Mark the choices for all plans that would be considered in pass 3.

i. Company JOIN (Application JOIN Engineer) (175,000 IOs)

ii. Company JOIN (Engineer JOIN Application) (150,000 IOs)
iii. Application JOIN (Company JOIN Engineer) (155,000 IOs)
iv. Application JOIN (Company JOIN Engineer) (160,000 IOs)
v. Engineer JOIN (Company JOIN Application) (215,000 IOs)
vi. (Company JOIN Application) JOIN Engineer(180,000 IOs)
vii. (Application JOIN Company) JOIN Engineer (200,000 IOs)
viii. (Application JOIN Engineer) JOIN Company (194,000 IOs)
ix. (Engineer JOIN Application) JOIN Company (195,000 IOs)
x. (Engineer JOIN Company) JOIN Application (165,000 IOs)

(j) (1.5 points) Mark the choices from the previous question for all plans that would be chosen at the end of pass3.

Solution:

(f) Pass 2, Pass 2, Pass 1, Pass 3, Pass 1. Note that the OR predicate is over 2 tables that have no associated join predicate, so the selection is postponed along with the cross-product, until after 3-way joins are done.

(g) i, ii, v, and vi will be considered because they are not cross products.

(h) ii and vi will be chosen because they have the lower cost for joining the two tables tables.

(i) Considers vi, viii

(j) Choses vi.

END OF PAPER